

NumPy Tutorial 9: NumPy and Pandas Together — Powerful Data Analysis

This chapter covers how NumPy and Pandas work together. After completing this tutorial, students will understand the connection between the two libraries and how to use both for effective data analysis.

1. Introduction

In previous tutorials, we learned NumPy deeply. Before that, we completed the full Pandas course. In this tutorial, we will bring both libraries together.

NumPy and Pandas are not separate tools — they are deeply connected. Understanding how they work together will make you a much stronger Data Scientist.

2. How Pandas Uses NumPy Internally

Pandas is built on top of NumPy. This means:

- Every Pandas Series is a NumPy array underneath.
- Every Pandas DataFrame stores its data using NumPy arrays.
- All Pandas operations use NumPy for actual calculations.

```
import pandas as pd
import numpy as np

series = pd.Series([10, 20, 30, 40, 50])
print(type(series.values))
```

Output:

```
<class 'numpy.ndarray'>
```

The values inside a Pandas Series are stored as a NumPy ndarray.

3. Converting Pandas Series to NumPy Array

```
import pandas as pd
import numpy as np
```

```
series = pd.Series([10, 20, 30, 40, 50])

arr = series.values
print(arr)
print(type(arr))
```

Output:

```
[10 20 30 40 50]
<class 'numpy.ndarray'>
```

We use .values to get the underlying NumPy array from a Pandas Series.

4. Converting Pandas DataFrame to NumPy Array

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Ali", "Sara", "Ahmed"],
    "Marks": [85, 90, 78],
    "Age": [22, 21, 23]
}

df = pd.DataFrame(data)

arr = df[["Marks", "Age"]].values
print(arr)
print(type(arr))
```

Output:

```
[[85 22]
 [90 21]
 [78 23]]
<class 'numpy.ndarray'>
```

We select only numeric columns and convert to NumPy array. This is very commonly done before feeding data to a Machine Learning model.

5. Creating Pandas DataFrame from NumPy Array

We can also go the other way — create a Pandas DataFrame from a NumPy array.

```
import numpy as np
import pandas as pd

np.random.seed(42)
arr = np.random.randint(50, 100, size=(5, 3))

df = pd.DataFrame(arr, columns=["Math", "Science", "English"])
print(df)
```

Output:

	Math	Science	English
0	77	88	75
1	60	72	91
2	85	55	63
3	70	94	81
4	68	79	58

This is very useful when we generate random data with NumPy and want to analyze it with Pandas.

6. Using NumPy Functions on Pandas Data

We can apply NumPy functions directly on Pandas columns.

```
import pandas as pd
import numpy as np

data = {"Marks": [55, 72, 68, 90, 45, 88, 76, 60, 95, 50]}
df = pd.DataFrame(data)

print("Mean:", np.mean(df["Marks"]))
print("Std:", np.std(df["Marks"]))
print("Max:", np.max(df["Marks"]))
print("Percentile 75:", np.percentile(df["Marks"], 75))
```

Output:

```
Mean: 69.9
Std: 16.42
Max: 95
Percentile 75: 85.0
```

NumPy functions work directly on Pandas Series because they are NumPy arrays internally.

7. Adding NumPy Calculated Columns to Pandas

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Ali", "Sara", "Ahmed", "Zara", "Umar"],
    "Marks": [85, 42, 78, 55, 90]
}

df = pd.DataFrame(data)

df["Normalized"] = (df["Marks"] - np.min(df["Marks"])) / (np.max(df["Marks"]) -
np.min(df["Marks"]))
df["Normalized"] = np.round(df["Normalized"], 2)

print(df)
```

Output:

	Name	Marks	Normalized
0	Ali	85	0.90
1	Sara	42	0.00
2	Ahmed	78	0.75
3	Zara	55	0.27
4	Umar	90	1.00

Normalization is a very common step in Machine Learning data preparation.

8. Using np.where with Pandas

```
import pandas as pd
import numpy as np
```

```
data = {"Name": ["Ali", "Sara", "Ahmed", "Zara", "Umar"],
        "Marks": [85, 42, 78, 55, 90]}

df = pd.DataFrame(data)

df["Grade"] = np.where(df["Marks"] >= 70, "Pass", "Fail")
print(df)
```

Output:

	Name	Marks	Grade
0	Ali	85	Pass
1	Sara	42	Fail
2	Ahmed	78	Pass
3	Zara	55	Fail
4	Umar	90	Pass

np.where works perfectly on Pandas columns and is much faster than using apply with a function.

9. Performance Comparison — Pandas vs NumPy

Sometimes NumPy operations are faster than Pandas for pure numerical calculations.

```
import pandas as pd
import numpy as np
import time

data = np.random.randint(1, 1000, size=1000000)
series = pd.Series(data)

start = time.time()
result = np.mean(data)
print("NumPy time:", time.time() - start)

start = time.time()
result = series.mean()
print("Pandas time:", time.time() - start)
```

For very large datasets with only numbers, NumPy is faster. For structured data with labels and mixed types, Pandas is more convenient.

10. When to Use NumPy vs Pandas

Situation	Use
Pure numerical calculations	NumPy
Data has labels and column names	Pandas
Data comes from CSV or Excel	Pandas
Preparing data for ML model	NumPy
Exploratory Data Analysis	Pandas
Mathematical operations on arrays	NumPy
Handling missing values	Pandas

In real projects, we use both together — Pandas for loading and cleaning data, NumPy for calculations and model preparation.

11. Conclusion of Tutorial 9

In this tutorial, students have learned:

1. How Pandas is built on top of NumPy.
2. How to convert Pandas Series and DataFrame to NumPy arrays.
3. How to create a Pandas DataFrame from a NumPy array.
4. How to apply NumPy functions on Pandas columns.
5. How to add NumPy calculated columns to a DataFrame.
6. How to use np.where with Pandas.
7. When to use NumPy vs Pandas.

In the next tutorial, we will apply everything we have learned in a complete real world data analysis project using NumPy.

This completes NumPy Tutorial 9.